

Project Executable Files

Date	2 july 2026
Team ID	xxxxxx (Team: Jaswanth Malapareddy, Madasu Rushi Venkata Ganesh, Rajani Meesala, Mounika Sonti, Nehan Chowdary Muvva)
Project Name	Opticrop-Smart Agricultural Production Optimization Engine
Maximum Marks	3 Marks

Project Executable Files This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA

5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	yes
9	Test files and test results	yes
10	Demo video or walkthrough (if required)	yes

Step 2: File / Folder Structure Provide an overview of your project's file and folder structure below:

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://opticrop-ai-463011982132.asia-southeast1.run.app
Login Credentials (Demo)	Login credentials are user name , password
Platform / Hosting Provider	Render.
Repository Link	https://github.com/MalapareddyJaswanth/opticrop_Ai.git
Demo Video Link	https://drive.google.com/file/d/1bkAYkAVqGf1d1aYtVBLvpMyLFvdPyjMH/view?usp=sharing

Step 4: Run Instructions Provide clear, step-by-step instructions for the evaluator to run your project locally:

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	1. Architectural & State Limitations • Stateless Operation: In its current production form, the application does not persist data. If a user refreshes the page, their entire generation history, selected inputs, and custom tailored copy are permanently lost. • Lack of User Authentication: The system lacks multi-tenant isolation. Without a database tier containing a Users table, it is impossible to implement personalized user dashboards, save favorite captions, or limit API usage per user.	When a system runs slow, crashes, or cannot handle data efficiently, the limitations are rectified through architectural changes: • Vertical Scaling (Upgrading): Adding more power (CPU, RAM, SSD) to a single machine to handle heavier workloads. • Horizontal Scaling (Outgrowing): Adding more machines to share the load, often managed by load balancers.

2	. API & Performance Constraints • Groq API Rate Limits: The backend relies entirely on a single API token for the Groq Cloud SDK. High concurrent traffic from multiple users can trigger rate limits (429 Too Many Requests), causing the application to stall. • Single-Point-of-Failure Dependency: Because the application does not cache previous generations or fall back to alternative LLM providers, any operational downtime on Groq's API directly results in a total outage of the caption generation service.	2. Machine Learning & Data Limitations If a machine learning model (like the OptiCrop engine) is inaccurate or biased, the limitations are rectified during the data and training phases: • Overfitting: When a model memorizes training data but fails on new data. Rectified by Regularization (penalizing complexity), adding Dropout layers, or gathering More diverse training data.
3	Data & Scalability Constraints • Stringent Input Lengths: The current UI and prompt structure limit user input descriptions. Extremely detailed product descriptions or nuanced brand guidelines may get truncated, resulting in less accurate outputs. • Normalized Data Overhead: While the proposed relational schema ensures data integrity, breaking down a single generation event into three separate tables (Users, Generations, Generated_Outputs) requires complex JOIN queries. As the historical log scales into millions of rows, this can introduce read latency without proper indexing.	No matter the project, teams generally rectify limitations by moving through these four distinct phases: **1. Identification & Logging:**Phase 1. Detecting the failure point through stress testing, user feedback, or error logs. The limitation must be clearly defined (e.g., "The system crashes when 1,000 users input soil data simultaneously").